

UTTEC vault Cross-Platform 이관 + Cloud 탑재 계획서

1. 배경 (Why)

1-1. vault 의 본질 재정의 (사용자 명시 2026-05-23)

"이 vault도 어떻게 보면, 다른 용도로 사용하기 전에 uttec이라는 회사를 기반으로 개발하고 있다고 생각해야합니다."

- 지금까지 vault = 개인 second-brain + 9 vault multi-agent hub (개인 운영 자산)
- 이제부터 vault = **UTTEC 회사 product candidate** + 회사 운영 hub
- 즉 vault 자체가 R&D 대상이며, "다른 시스템 동작 확인 → cloud 탑재" 까지 가야 product 수준

1-2. 현재 제약

- Windows 단일 PC 종속 (myWiki single-source 정책, `project_dual_pc.md`)
- skill-hook 의 58개 hardcoded `C:\todo\today\...` 경로 (12 SKILL 파일)
- PowerShell 명령 10건 (5 SKILL 파일)
- raw/ junction + memory junction = NTFS 전용 기능
- Claude project slug `C---todo-today` 가 PC별 다름 → memory 경로 mismatch 잠재

1-3. 이미 cross-platform 우호적 자산

- 6 hook 중 5개 Python (notion-sync, notion-cleanup, setup-memory-sync, check-raw-junctions, create-daily-report)
- search vault — FastAPI + Vite, Python + Node = OS-agnostic 표준
- 9 vault inbox 카드 frontmatter = YAML 표준
- git, Notion, Tailscale = cross-platform external dependencies

2. 목표 (Goal)

단계	목표	검증 기준
L1	Mac 개발 PC 에서 핵심 워크플로우 가동	<code>/work-start</code> + <code>/work-end</code> 무에러 실행, <code>_inbox</code> 카드 흡수 가능
L2	OS-agnostic 표준 layer 확립	Windows + macOS + Linux 동일 명령 동일 결과 (CI matrix 통과)
L3	Cloud 탑재 가능 architecture	Docker container 단일 배포 단위 + persistent volume + web UI 접근

2-1. Non-goal (이번 사이클에서 안 함)

- Windows 단일 source 정책 즉시 폐기 (L2 종료 시점에 재합의)
- 9 vault 전체 cloud 이관 (search 같은 vault 별 분리 진행)
- Notion 완전 대체 (Notion = view layer 유지, vault = source-of-truth)

3. 현재 Windows 종속성 인벤토리 (Phase 0 baseline)

3-1. 카테고리별 종속점

카테고리	종속점	정량	영향도	해소 난이도
A. 경로 hardcoding	C:\todo\today\...	58건 / 12 file	매우 높음	중 (sed-style 치환 + env var 도입)
B. PowerShell 종속	.ps1 + powershell -Command	10건 / 5 file + 1 .ps1 파일	높음	중 (Python 재작성)
C. NTFS Junction	raw/* + .claude/memory link	7+ junction	중간	낮음 (symlink로 OS detect 분기)
D. Claude project slug	C--todo-today 경로	1건 (memory junction)	중간	낮음 (slug 환경변수화)
E. External alias	ssh ubuntu/digital/shield	~5건 (skill 내 가이드 텍스트)	낮음	매우 낮음 (config 추가)
F. cmd/PowerShell profile	terminal 기본 디렉토리 registry AutoRun	1건 (PC-level 설정)	낮음	낮음 (Mac shell rc 동등 적용)

3-2. 기 cross-platform 자산 (재사용 가능)

- `setup-memory-sync.py` — 이미 Mac/Linux 분기 존재 (=== Memory Sync Setup 출력 헤더 분기)
- `notion-sync.py` / `notion-cleanup.py` — requests 기반 순수 Python
- `check-raw-junctions.py` — Windows junction 전용이지만 OS detect 분기 추가 용이
- `create-daily-report.py` — 순수 Python
- search vault `backend/` (FastAPI) + `frontend/` (Vite) — OS-agnostic
- 9 vault inbox 카드 YAML frontmatter — 형식 표준

4. Phase 계획 (단계적 진화)

Phase 0 — 인벤토리·진단 (0.5일)

산출물: `myWiki/second-brain/raw/vault-portability-audit/2026-05-23.md`

작업:

1. 자동 audit script `bin/audit-portability.py` 작성 (rg 기반)
 - `C:\todo\today` / `C:/todo/today` 출현 위치 전부 인벤토리화
 - `powershell` / `\.ps1` / `Get-ChildItem` / `Set-Content` 출현
 - junction 의존 경로 (`.claude/memory`, `myWiki/second-brain/raw/*`)
 - 환경 가정 (NTFS, registry, `$env:USERNAME` 등)
2. 결과를 카테고리 (A~F) 별 정리표 + file:line 인용
3. Phase 1 우선순위 결정 (영향도 × 해소 난이도)

완료 조건: audit script 가 Mac/Linux 에서도 실행 가능 + 결과 표 작성

Phase 1 — 경로 추상화 layer (1~2일)

산출물: `bin/vault-config.py` + `.claude/config/vault-paths.json`

작업:

1. `VAULT_ROOT` 환경변수 도입 (기본값: 자동 detect — git rev-parse 또는 마커 파일)
2. `vault-paths.json` 단일 진입점:

```
{
  "vault_root": "${VAULT_ROOT}",
  "sessions_dir": "${VAULT_ROOT}/.claude/sessions",
  "memory_dir": "${VAULT_ROOT}/.claude/memory",
  "mywiki_root": "${VAULT_ROOT}/myWiki",
  "second_brain": "${VAULT_ROOT}/myWiki/second-brain",
  "inbox_pending": "${VAULT_ROOT}/myWiki/_inbox/pending",
  "work_report": "${VAULT_ROOT}/작업보고서"
}
```

3. 모든 hook 에서 hardcoded 경로 제거, `vault_config.get('memory_dir')` 패턴 사용
4. SKILL.md 의 bash 블록은 일단 OS-detect 분기 `if Windows: ... else: ...` (Phase 2 에서 통합)

완료 조건: 6 hook 전부 `vault-config` 사용 + `audit-portability` 실행 시 hook 측 경로 인용 0건

Phase 2 — Shell layer 통합 (2~3일)

산출물: `bin/work_start.py` + `bin/work_end.py` + `bin/vault_lint.py` (PowerShell `.lint-script.ps1` 대체)

작업:

1. `.lint-script.ps1` → `bin/vault_lint.py` 재작성 (Python rg/glob)
2. `log-archive/_check-size.ps1` → `bin/log_archive_check.py`
3. `setup-memory-sync.ps1` 폐기 (`setup-memory-sync.py` 이미 Mac/Linux 지원)
4. SKILL.md 의 PowerShell 블록 → `python bin/<cmd>.py` 호출로 통합
5. Windows / macOS / Linux 모두 `python` 실행 (필요시 `python3` alias)

완료 조건: `.ps1` 파일 0건, 모든 SKILL 명령이 `python bin/...` 형태

Phase 3 — OS 분기 hook 표준화 (1일)

산출물: `bin/link_manager.py` (junction/symlink 추상화)

작업:

1. `link_manager.create(src, target)` — Windows 면 `mklink /J`, macOS/Linux 면 `os.symlink`
2. `link_manager.verify(path)` — junction/symlink 둘 다 정합성 체크
3. `check-raw-junctions.py` 를 `link_manager` 기반으로 재작성
4. `setup-memory-sync.py` 의 OS 분기 코드 통일

5. raw/ 디렉토리 link 자동 복구 hook 추가

완료 조건: Mac 에서 raw/ 디렉토리가 정상 link 로 생성됨

Phase 4 — Mac dry-run (read-only) (0.5일)

산출물: myWiki/second-brain/raw/vault-portability-audit/2026-05-XX_mac_dryrun.md

작업:

1. ssh ubuntu (현 개발 PC, Linux 지만 Mac 과 유사 패턴) 또는 신규 Mac 에 git clone
2. `python bin/setup-memory-sync.py` → memory junction 생성 확인
3. `python bin/work_start.py` 시험 실행 (write 부분 dry-run 플래그)
4. 차단점 / 미해결 OS 분기 인벤토리화
5. Phase 1~3 회귀 패치 (필요 시)

완료 조건: `/work-start` 가 Mac 에서 무에러로 완주 (write 제외)

Phase 5 — Mac write-mode + 정책 합의 (1~2일)

산출물: MEMORY.md 정책 갱신 + 신규 메모리 `project_multi_pc_sync.md`

작업:

1. `project_dual_pc.md` 메모리 갱신 — "Windows = single source / Mac = development replica" 정책 명문화
2. 이중 write 차단:
 - 옵션 A: file lock (`.vault.lock` 마커, Mac 측 write 시도 시 경고)
 - 옵션 B: git branch 분리 (`main` = Windows / `dev-mac` = Mac, PR 로 머지)
 - 옵션 C: Mac 은 search vault 같은 sub-vault 만 write
3. 9 vault PROTOCOL.md 동기화는 **단일 PC 만** (Windows 정본 유지)
4. Notion sync 도 단일 PC 만 (이중 발동 시 race condition)
5. 사용자 broker 패턴: Mac 작업 결과는 inbox 카드 발송 → Windows 측 흡수

완료 조건: Mac 측 write 1주 dogfooding 후 정책 준수율 100%

Phase 6 — 자동 회귀 테스트 (1~2일)

산출물: `.github/workflows/vault-portability.yml` + `bin/test_portability.py`

작업:

1. `bin/test_portability.py` — 핵심 hook 들 sanity check
 - vault-paths.json 정합성
 - link_manager 의 create/verify 왕복
 - work-start dry-run
 - audit-portability 자가 검증
2. GitHub Actions matrix: `windows-latest` + `macos-latest` + `ubuntu-latest`
3. CI 통과 = portability 회귀 0 보장

4. PR template 에 "portability impact 체크박스" 추가

완료 조건: CI green 3 OS + 회귀 시 자동 차단

Phase 7 — Cloud 탑재 (3~5일, L3 목표)

산출물: `Dockerfile` + `docker-compose.yml` + `deploy/cloud-deploy.md`

작업:

1. Container 화
 - base: `python:3.12-slim`
 - vault root mount (persistent volume)
 - 필요 시 Node (search vault 용)
2. Secret 관리
 - Notion token / Claude API key / Tailscale auth = env var 또는 Docker secret
3. Persistent volume
 - `myWiki/second-brain/` (지식)
 - `myWiki/_inbox/` (메시지큐)
 - `작업보고서/` (운영 기록)
4. Web UI
 - search vault FastAPI 패턴 재사용 — vault 전체 search/edit web UI
 - 인증: Cloudflare Access 또는 OAuth (기존 Notion·Calendar OAuth 패턴 활용)
5. Cloud target 결정 — DigitalOcean (기존 인프라 정합) / AWS EC2 (ec2-remote skill 기존) / GCP
6. Multi-agent inbox 진화
 - 단순 git polling → 메시지큐 (SQS / DO Spaces 이벤트) 검토
 - 단, git 기반 audit trail 유지

완료 조건: cloud 인스턴스에서 `/work-start` + 카드 흡수 + commit 까지 사이클 정상 작동

5. 총 일정 추정

Phase	작업량	누적
0	0.5일	0.5일
1	1~2일	1.5~2.5일
2	2~3일	3.5~5.5일
3	1일	4.5~6.5일
4	0.5일	5~7일
5	1~2일	6~9일
6	1~2일	7~11일
7	3~5일	10~16일

→ L1 (Mac dry-run) = **~7일 후**, L2 (CI green) = **~11일 후**, L3 (cloud) = **~16일 후**

6. 위험 / Tradeoffs

위험	영향	완화
9 vault PROTOCOL 영향	8/9 동기화 정책에 Mac/cloud 추가 → 10/11 vault 동기화 부담	Phase 5 정책 합의 시 search vault 패턴 (단일 source) 차용
vault scope 격리 정책 흔들림	mywiki-claude 가 다른 PC vault 진입 가능성	격리는 Phase 5 정책에서 "PC 경계" 기준 재정의
Notion sync race condition	이중 PC 동시 발동 시 카드 중복	Phase 5: 단일 PC 만 sync 권한
Cloud 보안 risk	API key / SSH key / 사용자 데이터 노출	Phase 7: secret manager + Cloudflare Access + audit log
회사 자산화 책임	"uttec product" 명명 시 IP/저작권 정리 필요	Phase 7 직전 사용자 결단 — open-source / 상용 / 사내 전용
이관 중 9 vault 운영 marginal cost	Phase 1~6 동안 두 PC 동시 운영 부담	Phase 4~5 사이 1~2주 dogfooding 기간 명시

7. 결정 필요 사항 (사용자 결단)

#	결정 항목	옵션	Claude 권장
D1	작업 슬롯 우선순위	(a) 즉시 Phase 0 시작 / (b) #18-#21-#24 후순 / (c) 다음 주 시작	(b) — 외부 카드 흡수 인프라 복구 먼저, Phase 0 는 next-next
D2	Mac 실체	(a) ssh ubuntu 활용 (Linux) / (b) 신규 Mac 도입 / (c) 둘 다 (matrix)	(a) — 기존 자산 활용. Mac 도입은 L3 단계에서 재검토
D3	Cloud target	(a) DigitalOcean / (b) AWS EC2 / (c) GCP / (d) 자체 VPS	(a) — 기존 인프라 정합 (uttecHome 7777 / search 등)
D4	정책 모델 (Phase 5)	(a) Windows single-source 유지 / (b) PC별 분기 source / (c) git PR 머지	(a) — 9 vault PROTOCOL 비대칭 최소화
D5	회사 자산화 범위	(a) UTTEC 사내 전용 / (b) GitHub public / (c) 상용 SaaS 후보	(a) 우선 → L3 도달 후 (c) 검토
D6	위임 모델	(a) mywiki-claude 단독 / (b) search-claude 일부 위임 / (c) 신규 portability-claude	(b) — Phase 7 web UI 는 search-claude 합리

8. 즉시 시작 가능한 첫 행동 (사용자 승인 시)

- Phase 0 audit script 작성** (2~3시간) — `bin/audit-portability.py` 신설, 결과 `myWiki/second-brain/raw/vault-portability-audit/2026-05-23_baseline.md`
- 메모리 박제** — `project_vault_portability.md` 신설 (본 계획서 요지)
- myWiki entity 신설** — `entities/vault-portability.md` (장기 추적 대상)
- ai-direction 판단 로그** — "vault = UTTEC product candidate 재정의"
- 작업보고서 5/23 신규 todo 행 추가** — Phase 0 시작 trigger

9. 참조

vault 내부

- `.claude/memory/project_dual_pc.md` — 현재 PC 역할 분리 정책
- `.claude/memory/reference_uttec_ubuntu_mac.md` — Ubuntu 개발 PC 셋업
- `.claude/memory/project_3vault_분리.md` — 5-vault / 9 Claude 구조
- `.claude/memory/feedback_mywiki_main_vault_role.md` — main vault hub 역할
- `myWiki/_inbox/PROTOCOL.md` — 9 vault 통신 표준
- `myWiki/second-brain/CLAUDE.md` — raw/junction 스키마

외부 참고

- search vault (`C:\todo\search\`) — FastAPI+Vite cross-platform 우호 사례
- uttecHome 7777 DigitalOcean 보류 (`project_uttechome_deploy_hold.md`) — Phase 7 cloud target 후보

변경 이력

- 2026-05-23: 초안 작성 (mywiki-claude, 사용자 지시)